Lab 7: Fuzzing for Input Validation Bugs

Name: Rishitth G R SRN: PES1UG24CS376 Section: G

Step 2: Running the buggy processor.py

```
PS C:\College Stuff\Lab & Mini Projects\SE-Labs-PES1UG24CS376\Lab 7> python processor.py
==== Running parse_storage_quota ====
Enter quota (e.g. 500MB, 10GB, 2TB): abc
Traceback (most recent call last):
  File "C:\College Stuff\Lab & Mini Projects\SE-Labs-PES1UG24CS376\Lab 7\processor.py", line
 56, in <module>
    main()
  File "C:\College Stuff\Lab & Mini Projects\SE-Labs-PES1UG24CS376\Lab 7\processor.py", line
 45, in main
    print(parse_storage_quota(q_input))
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\College Stuff\Lab & Mini Projects\SE-Labs-PES1UG24CS376\Lab 7\processor.py", line
 9, in parse_storage_quota
    value = int(val_str)
            ^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'A'
PS C:\College Stuff\Lab & Mini Projects\SE-Labs-PES1UG24CS376\Lab 7>
```

Step 4: test_processor.py on the buggy code (3 tests fail)

```
PS C:\College Stuff\Lab & Mini Projects\SE-Labs-PES1UG24CS376\Lab 7> python -m pytest test_p
rocessor.py
========================= test session starts =========================
platform win32 -- Python 3.12.10, pytest-9.2.0, pluggy-1.7.0
rootdir: C:\College Stuff\Lab & Mini Projects\SE-Labs-PES1UG24CS376\Lab 7
plugins: hypothesis-6.100.0, platformdirs-4.14.0
collected 3 items

test_processor.py FFF                                            [100%]

============================== FAILURES ===============================
_________________ test_parse_storage_quota_no_crash _________________
ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
 AssertionError: parse_storage_quota crashed with: KeyError('00') for input '000'
 Falsifying example: test_parse_storage_quota_no_crash(s='000')
 AssertionError: ... ValueError("invalid literal for int() with base 10: ''") for input ''
 Falsifying example: test_parse_storage_quota_no_crash(s='')
 AssertionError: ... AttributeError("'NoneType' object has no attribute 'strip'") for input
None
 Falsifying example: test_parse_storage_quota_no_crash(s=None)
_____________ test_compute_compression_ratio_safe _____________
ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
 AssertionError: ... ValueError("could not convert string to float: ''") for input ':'
 Falsifying example: test_compute_compression_ratio_safe(s=':')
 AssertionError: ... ValueError('not enough values to unpack (expected 2, got 1)') for input
 ''
 Falsifying example: test_compute_compression_ratio_safe(s='')
 AssertionError: ... AttributeError("'NoneType' object has no attribute 'split'") for input
None
 Falsifying example: test_compute_compression_ratio_safe(s=None)
_____________ test_lookup_bandwidth_tier_rate_safe _____________
ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
 AssertionError: ... KeyError('') for input ''
 Falsifying example: test_lookup_bandwidth_tier_rate_safe(s='')
 AssertionError: ... AttributeError("'NoneType' object has no attribute 'strip'") for input
None
 Falsifying example: test_lookup_bandwidth_tier_rate_safe(s=None)
======================= short test summary info =======================
FAILED test_processor.py::test_parse_storage_quota_no_crash - ExceptionGroup: Hypothesis fou
nd 3 distinct failures. (3 sub-exceptions)
FAILED test_processor.py::test_compute_compression_ratio_safe - ExceptionGroup: Hypothesis f
ound 3 distinct failures. (3 sub-exceptions)
FAILED test_processor.py::test_lookup_bandwidth_tier_rate_safe - ExceptionGroup: Hypothesis
found 2 distinct failures. (2 sub-exceptions)
======================== 3 failed in 10.23s ========================
PS C:\College Stuff\Lab & Mini Projects\SE-Labs-PES1UG24CS376\Lab 7>
```

Step 5: Running the fixed processor.py

```
PS C:\College Stuff\Lab & Mini Projects\SE-Labs-PES1UG24CS376\Lab 7> python processor.py
==== Running parse_storage_quota ====
Enter quota (e.g. 500MB, 10GB, 2TB): abc
Invalid quota

==== Running compute_compression_ratio ====
Enter raw:compressed bytes (e.g. 1024:256): 5:0
Compression Ratio: Invalid input

==== Running lookup_bandwidth_tier_rate ====
Enter tier descriptor (e.g. HOT_TIER, COLD, ARCHIVE): xyz
Rate per GB: $ Unknown tier
PS C:\College Stuff\Lab & Mini Projects\SE-Labs-PES1UG24CS376\Lab 7>
```

Step 6: test_processor.py on the fixed code (3 tests pass)

```
PS C:\College Stuff\Lab & Mini Projects\SE-Labs-PES1UG24CS376\Lab 7> python -m pytest test_p
rocessor.py
========================= test session starts =========================
platform win32 -- Python 3.12.10, pytest-9.2.0, pluggy-1.7.0
rootdir: C:\College Stuff\Lab & Mini Projects\SE-Labs-PES1UG24CS376\Lab 7
plugins: hypothesis-6.100.0, platformdirs-4.14.0
collected 3 items

test_processor.py ...                                            [100%]

========================== 3 passed in 1.08s ==========================
PS C:\College Stuff\Lab & Mini Projects\SE-Labs-PES1UG24CS376\Lab 7>
```

Fixed processor.py

```
import math
import re


def parse_storage_quota(quota_str):
    """
    Parses storage quota string with unit suffix into Gigabytes (GB).
    Format: '<digits><2-letter unit>' (e.g. '500MB', '20GB', '2TB').
    Returns 'Invalid quota' for any malformed input.
    """
    if not isinstance(quota_str, str):
        return "Invalid quota"
    match = re.fullmatch(r"([0-9]+)(MB|GB|TB)", quota_str.strip().upper(), re.ASCII)
    if not match:
        return "Invalid quota"
    multipliers = {"MB": 0.001, "GB": 1.0, "TB": 1000.0}
    try:
        gb_total = int(match.group(1)) * multipliers[match.group(2)]
    except (ValueError, OverflowError):
        return "Invalid quota"
    return f"Quota: {gb_total:.2f} GB"


def compute_compression_ratio(size_metric_str):
    """
    Computes compression ratio: raw_bytes / compressed_bytes.
    Format: 'raw_bytes:compressed_bytes' (e.g. '1024:256').
    Returns None for malformed input or a zero/invalid compressed size.
    """
    if not isinstance(size_metric_str, str):
        return None
    parts = size_metric_str.split(":")
    if len(parts) != 2:
        return None
    try:
        raw_bytes = float(parts[0])
        comp_bytes = float(parts[1])
    except ValueError:
        return None
    if not (math.isfinite(raw_bytes) and math.isfinite(comp_bytes)):
        return None
    if raw_bytes < 0 or comp_bytes <= 0:
        return None
    try:
        return round(raw_bytes / comp_bytes, 2)
    except OverflowError:
        return None


def lookup_bandwidth_tier_rate(tier_descriptor):
    """
    Looks up rate per GB for egress traffic using the first 3 letters of the tier name.
    Returns None for non-string input or an unknown tier.
    """
    if not isinstance(tier_descriptor, str):
        return None
    prefix = tier_descriptor.strip().upper()[:3]
    tier_rates = {"HOT": 0.08, "COL": 0.05, "ARC": 0.02}
    return tier_rates.get(prefix)


def main():
    print("==== Running parse_storage_quota ====")
    q_input = input("Enter quota (e.g. 500MB, 10GB, 2TB): ")
    print(parse_storage_quota(q_input))

    print("\n==== Running compute_compression_ratio ====")
    c_input = input("Enter raw:compressed bytes (e.g. 1024:256): ")
    ratio = compute_compression_ratio(c_input)
    print("Compression Ratio:", ratio if ratio is not None else "Invalid input")

    print("\n==== Running lookup_bandwidth_tier_rate ====")
    t_input = input("Enter tier descriptor (e.g. HOT_TIER, COLD, ARCHIVE): ")
    rate = lookup_bandwidth_tier_rate(t_input)
    print("Rate per GB: $", rate if rate is not None else "Unknown tier")


if __name__ == "__main__":
    main()
```

Updated test_processor.py

```
from hypothesis import given, strategies as st
from processor import parse_storage_quota, compute_compression_ratio, lookup_bandwidth_tier_rate


@given(st.text() | st.none())
def test_parse_storage_quota_no_crash(s):
    try:
        result = parse_storage_quota(s)
    except Exception as e:
        assert False, f"parse_storage_quota crashed with: {e!r} for input {s!r}"
    assert isinstance(result, str)


@given(st.text() | st.none())
def test_compute_compression_ratio_safe(s):
    try:
        result = compute_compression_ratio(s)
    except Exception as e:
        assert False, f"compute_compression_ratio crashed with: {e!r} for input {s!r}"
    assert result is None or isinstance(result, float)


@given(st.text() | st.none())
def test_lookup_bandwidth_tier_rate_safe(s):
    try:
        result = lookup_bandwidth_tier_rate(s)
    except Exception as e:
        assert False, f"lookup_bandwidth_tier_rate crashed with: {e!r} for input {s!r}"
    assert result is None or result in (0.08, 0.05, 0.02)
```